

Artificial Intelligence Reinforcement Learning

Session 3
Learning Through Interaction

Ali Ismail-Fawaz

ENSISA, Université Haute-Alsace

Why Do We Need Reinforcement Learning?

What We Have Learned So Far

In the previous sessions, we studied algorithms that allow an intelligent agent to **plan** before acting.

- ▶ Search algorithms (BFS, DFS, A*) compute a path to a goal.
- ▶ Game-playing algorithms (Minimax, MCTS) compute the best move.
- ▶ The agent decides what to do *before* interacting with the environment.

Real-World Problems Are Different

In many applications, the best strategy is **not known in advance**.

The agent must:

- ▶ interact with its environment,
- ▶ observe the consequences of its actions,
- ▶ improve its behavior through experience.

Contents

Foundations of Reinforcement Learning

Finding the Optimal Policy

Learning Without a Model

Learning Through Interaction

Human Learning

How does a child learn to ride a bicycle?

- ▶ Try
- ▶ Fall
- ▶ Adjust
- ▶ Try again

The child gradually improves through **trial and error**.

Artificial Intelligence

Many intelligent systems learn in a similar way:

- ▶ Robots learning to walk
- ▶ Autonomous driving
- ▶ AlphaGo improving through self-play

Instead of being programmed with the best strategy, they **learn from experience**.

Key Question

How can an agent learn good decisions through interaction with its environment?

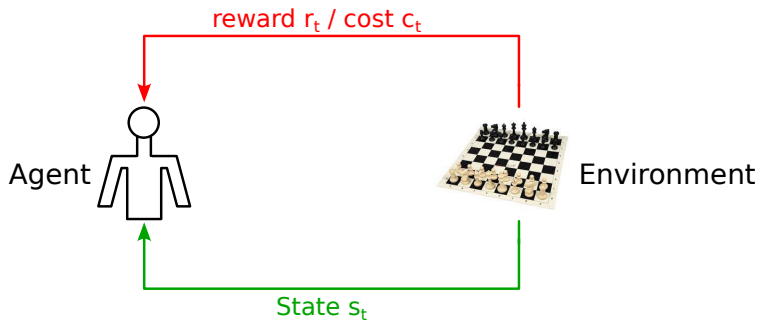
Agent–Environment Interaction



Environment

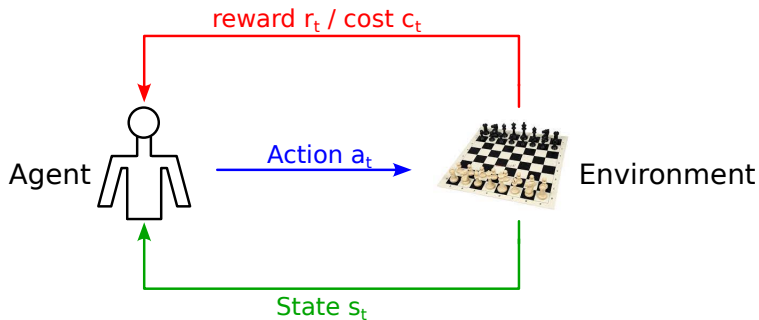
- ▶ An **agent** interacts with an **environment**.

Agent–Environment Interaction



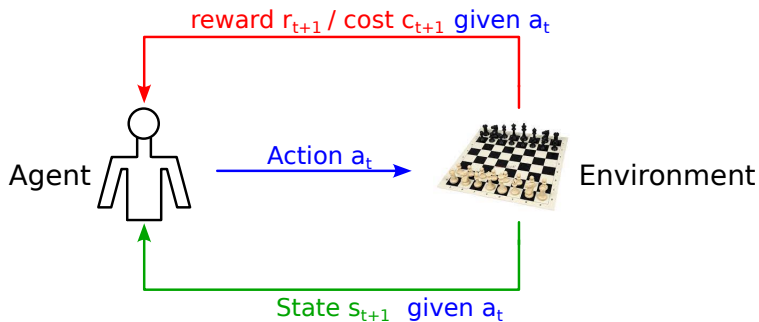
- ▶ An **agent** interacts with an **environment**.
- ▶ The environment provides an **observation** and a **reward**.

Agent–Environment Interaction



- ▶ An **agent** interacts with an **environment**.
- ▶ The environment provides an **observation** and a **reward**.
- ▶ The agent chooses an **action**.

Agent–Environment Interaction



- ▶ An **agent** interacts with an **environment**.
- ▶ The environment provides an **observation** and a **reward**.
- ▶ The agent chooses an **action**.
- ▶ The interaction repeats over time, allowing the agent to improve its behavior.

States, Actions, and Rewards

The Three Fundamental Concepts

At each interaction, the agent receives information about the environment, makes a decision, and receives feedback.

State (s) A description of the current situation of the environment.

Action (a) A decision chosen by the agent that may change the state of the environment.

Reward (r) A scalar feedback indicating how good or bad the chosen action was. In some applications, this feedback is expressed as a **cost (c)** to be minimized instead of a reward to be maximized.

Objective

The agent must learn a strategy that maximizes the **total reward** collected over time, rather than the reward from a single action.

An Episode of Interaction

Sequential Decision Making

Reinforcement learning consists of a sequence of interactions between the agent and the environment. Each action influences future states and future rewards.

$$\underbrace{s_0 \rightarrow a_0 \rightarrow r_1, s_1 \rightarrow a_1 \rightarrow r_2, s_2 \rightarrow a_2 \rightarrow \dots \rightarrow s_T}_{\text{Episode}}$$

$$s_1 \rightarrow \underbrace{a_1 \rightarrow r_2, s_2 \rightarrow a_2}_{\text{One Time Step}}$$

- ▶ **Time Step:** A single interaction between the agent and the environment.
- ▶ **Episode:** A complete sequence of interactions from the initial state until a terminal state.

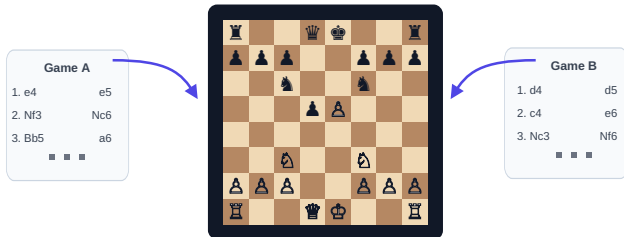
The Markov Property

Definition

A process satisfies the **Markov Property** if the future depends only on the **current state**, not on the complete history.

$$P(s_{t+1} \mid s_t, a_t) = P(s_{t+1} \mid s_0, a_0, \dots, s_t, a_t)$$

The Markov Property Through Chess
Different histories can lead to the same current state



Key Idea

A **well-defined state** contains all the information needed to predict the next state.

If information from the past still affects the future, it should be included in the state.

Markov Decision Processes

Definition

A **Markov Decision Process (MDP)** is a mathematical framework used to model sequential decision-making problems that satisfy the Markov Property.

An MDP is defined by the tuple

$$(S, A, P, R, \gamma)$$

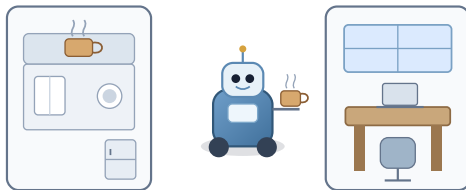
S	Set of possible states
A	Set of possible actions
P	Transition probabilities
R	Reward (or cost) function
γ	Discount factor

Example: Coffee Delivery Robot

Scenario

A mobile robot delivers coffee inside an office. Its objective is to pick up a cup of coffee from the kitchen and deliver it to an employee's office while minimizing the delivery time.

- ▶ Where is the robot?
- ▶ Is it carrying coffee?
- ▶ Which action should it take next?
- ▶ Will the action always succeed?



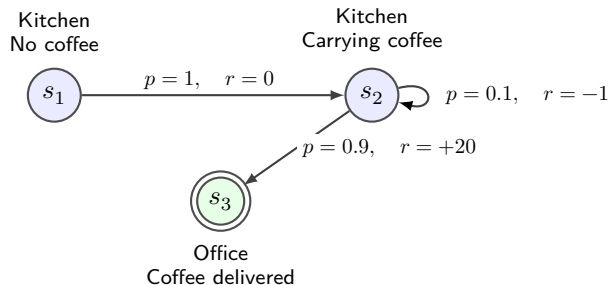
Observation

This problem involves making a sequence of decisions under uncertainty, making it a natural **Markov Decision Process (MDP)**.

Modeling the Coffee Robot as an MDP

Scenario

The robot must pick up coffee from the kitchen and deliver it to an office. Its movement may occasionally fail.



How to Read It

- ▶ Nodes represent **states**.
- ▶ The agent chooses an **action**.
- ▶ An action may produce different outcomes.
- ▶ Each arrow gives:
 - ▶ transition probability p ,
 - ▶ immediate reward r .
- ▶ The double circle represents a **terminal state**.

Actions

- a_1 : Pick up the coffee
- a_2 : Move toward the office

Policies

Definition

A **policy** defines the agent's behavior by specifying which action should be taken in every state.

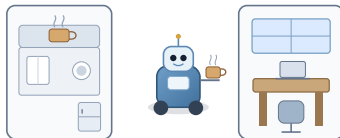
A policy is commonly denoted by

$$\pi(a \mid s)$$

which represents the probability of selecting action a when the agent is in state s .

Coffee Robot Example

State	Action chosen
Kitchen, no coffee	Pick up coffee
Kitchen, carrying coffee	Move to office
Office, coffee delivered	Stop



Key Idea

A policy is simply a **mapping from states to actions**.

Deterministic and Stochastic Policies

Two Ways to Define a Policy

A policy may select one action with certainty or assign probabilities to several possible actions.

Deterministic Policy

The policy selects exactly one action in each state:

$$\pi(s) = a$$

Coffee Robot Example

If the robot is carrying coffee, always move toward the office.

Stochastic Policy

The policy assigns a probability to each action:

$$\pi(a \mid s) = P(A_t = a \mid S_t = s)$$

with

$$\sum_{a \in A} \pi(a \mid s) = 1.$$

Coffee Robot Example

$$\pi(\text{Left} \mid s) = 0.8, \quad \pi(\text{Right} \mid s) = 0.2.$$

Returns: Measuring the Quality of a Policy

Question

How can we compare two different policies?

Suppose the coffee robot receives the following rewards:

$$+20, \quad -1, \quad -1, \quad +20$$

where:

- ▶ +20 for successfully delivering a coffee,
- ▶ -1 for every movement.

The total reward accumulated from time step t is called the **return**:

$$G_t = R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \dots$$

Key Idea

The objective of reinforcement learning is to find a policy that maximizes the **expected return**.

The Discount Factor

Definition

The **discount factor** $\gamma \in [0, 1]$ determines how much the agent values future rewards compared to immediate rewards.

$$G_t = R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \dots$$

Small γ (e.g. 0.2)

- ▶ Focus on immediate rewards.
- ▶ Short-term decisions.

Large γ (e.g. 0.99)

- ▶ Future rewards remain important.
- ▶ Long-term planning.

The Discount Factor

Definition

The **discount factor** $\gamma \in [0, 1]$ determines how much the agent values future rewards compared to immediate rewards.

$$G_t = R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \dots$$

Small γ (e.g. 0.2)

- ▶ Focus on immediate rewards.
- ▶ Short-term decisions.

Large γ (e.g. 0.99)

- ▶ Future rewards remain important.
- ▶ Long-term planning.

Analogy

Suppose you have an exam next week.

Would you rather:

- ▶ relax and play video games tonight, or
- ▶ spend the evening studying to achieve a better grade later?

A larger discount factor values the future more, making the second choice more attractive.

State Value Function

Question

If the robot starts from a particular state, how good is that state?

The **state-value function** measures the expected return obtained when starting from state s and following a policy π :

$$V^{\pi}(s) = \mathbb{E}_{\pi}[G_t \mid S_t = s]$$

Coffee Robot Example

- ▶ Kitchen, no coffee
- ▶ Kitchen, carrying coffee
- ▶ Coffee delivered

State	Value
s_1	14.2
s_2	18.5
s_3	20.0

Return vs. Value Function

Return (G_t)

Computed from one trajectory
Starts at a specific time step
Can vary from one episode to another

Value Function ($V^{\pi}(s)$)

Average over all possible future trajectories
Associated with a state
Represents the expected quality of a state

Contents

Foundations of Reinforcement Learning

Finding the Optimal Policy

Learning Without a Model

Definition: Principle of Optimality

Bellman's Principle of Optimality

An optimal policy has the property that, after any transition, the remaining decisions must also be optimal from the resulting state onward.

Implications

- ▶ Every subproblem is itself optimal.
- ▶ Enables recursive solutions.
- ▶ Foundation of Dynamic Programming.

Example

If the optimal path is

$$A \rightarrow B \rightarrow C \rightarrow D,$$

then

$$B \rightarrow C \rightarrow D$$

is also optimal.

Algorithmic Consequence

Once the optimal policy is reached,

- ▶ Policy improvement returns the same policy.
- ▶ Value updates no longer modify the value function.

Both algorithms have converged.

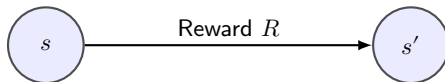
Bellman's Principle of Optimality

Key Observation

The value of a state can be decomposed into:

- ▶ the **immediate reward** received after taking an action,
- ▶ the **value of the next state**.

$$\text{Value} = \text{Immediate Reward} + \text{Future Value}$$



Current state

Next state



Richard Bellman

1920–1984

American mathematician who introduced **Dynamic Programming** and Bellman's Principle of Optimality.

Key Idea

Instead of evaluating an entire future trajectory, we can evaluate it recursively, **one step at a time**.

The Bellman Expectation Equation

Evaluating a Policy

Bellman's Principle states that the value of a state is equal to the expected immediate reward plus the discounted value of the next state.

$$V^{\pi}(s) = \mathbb{E}_{\pi} [R_{t+1} + \gamma V^{\pi}(S_{t+1}) \mid S_t = s]$$

Immediate Reward

$$R_{t+1}$$

The reward received after taking an action from the current state.

Future Value

$$\gamma V^{\pi}(S_{t+1})$$

The discounted value of the next state reached after the transition.

Why an Expectation?

Both the chosen action and the next state may be uncertain. Therefore, we consider the **expected** future return under policy π .

Deriving the Bellman Expectation Equation

Derivation

Starting from the definition of the value function,

$$V^\pi(s) = \mathbb{E}_\pi [G_t \mid S_t = s].$$

Expanding the return gives

$$\begin{aligned} V^\pi(s) &= \mathbb{E}_\pi [R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \dots \mid S_t = s] \\ &= \mathbb{E}_\pi [R_{t+1} + \gamma (R_{t+2} + \gamma R_{t+3} + \dots) \mid S_t = s] \\ &= \mathbb{E}_\pi [R_{t+1} + \gamma G_{t+1} \mid S_t = s] \\ &= \mathbb{E}_\pi [R_{t+1} + \gamma V^\pi(S_{t+1}) \mid S_t = s]. \end{aligned}$$

Why Does the Last Step Hold?

By definition,

$$V^\pi(S_{t+1}) = \mathbb{E}_\pi [G_{t+1} \mid S_{t+1}].$$

Therefore,

$$\begin{aligned} \mathbb{E}_\pi [V^\pi(S_{t+1}) \mid S_t = s] &= \mathbb{E}_\pi [\mathbb{E}_\pi [G_{t+1} \mid S_{t+1}] \mid S_t = s] \\ &= \mathbb{E}_\pi [G_{t+1} \mid S_t = s]. \end{aligned}$$

This uses the **law of iterated expectations**:

$$\mathbb{E} [\mathbb{E}[X \mid Y] \mid Z] = \mathbb{E}[X \mid Z].$$

Bellman Expectation Equation

$$V^\pi(s) = \mathbb{E}_\pi [R_{t+1} + \gamma V^\pi(S_{t+1}) \mid S_t = s]$$

Expanding the expectation over actions and next states gives

$$V^\pi(s) = \sum_{a \in A} \pi(a \mid s) \sum_{s' \in S} P(s' \mid s, a) [R(s, a, s') + \gamma V^\pi(s')].$$

The Bellman Optimality Equation

From Policy Evaluation to Policy Optimization

The Bellman expectation equation evaluates a **fixed policy** π . However, in reinforcement learning, our objective is to find the **optimal policy** π^* that maximizes the expected return.

Bellman Expectation Equation

$$V^\pi(s) = \sum_a \pi(a|s) \sum_{s'} P(s'|s, a) [R(s, a, s') + \gamma V^\pi(s')]$$

Average over actions chosen by the policy.

Bellman Optimality Equation

$$V^*(s) = \max_a \sum_{s'} P(s'|s, a) [R(s, a, s') + \gamma V^*(s')]$$

Select the action with the highest expected value.

Key Difference

Instead of following a fixed policy and averaging over its actions, the Bellman optimality equation directly selects the action that maximizes the expected return.

Why Do We Need Dynamic Programming?

Can We Solve the Bellman Equation Directly?

For a fixed policy π , the Bellman expectation equation forms a system of linear equations:

$$V^\pi = R^\pi + \gamma P^\pi V^\pi$$

or equivalently,

$$(I - \gamma P^\pi)V^\pi = R^\pi.$$

Direct Solution

$$V^\pi = (I - \gamma P^\pi)^{-1} R^\pi$$

Challenges

- ▶ Large state spaces.
- ▶ Matrix inversion is computationally expensive.
- ▶ Often infeasible for realistic problems.

Key Idea

Instead of solving a large system of equations directly, Dynamic Programming computes the solution iteratively.

Dynamic Programming: Two Fundamental Operations

Core Idea

Dynamic Programming finds an optimal policy by repeatedly applying two complementary operations.

1. Policy Evaluation

- ▶ Assume the current policy π is fixed.
- ▶ Compute the value function V^π .
- ▶ Estimate how good each state is when following π .

$V^\pi \Leftarrow \text{Evaluate } \pi$

2. Policy Improvement

- ▶ Use V^π to improve the policy.
- ▶ Select the action with the highest expected return.
- ▶ Obtain a new policy π' .

$\pi' \Leftarrow \text{Improve using } V^\pi$

Key Idea

Dynamic Programming alternates between **evaluating the current policy** and **improving it** until no further improvement is possible.

Policy Evaluation

Objective

Given a fixed policy π , estimate the value function $V^\pi(s)$ for every state.

$$V_{k+1}(s) = \sum_a \pi(a|s) \sum_{s'} P(s'|s, a) [R(s, a, s') + \gamma V_k(s')]$$

Initialization

- ▶ Start with any values.
- ▶ Common choice:

$$V_0(s) = 0$$

Iteration

- ▶ Apply the Bellman update.
- ▶ Repeat until values stop changing.

Key Idea

Policy evaluation repeatedly applies the Bellman expectation equation until the value estimates converge.

Policy Improvement

Objective

Given the value function V^π , construct a better policy by choosing the action with the highest expected return in every state.

$$\pi'(s) = \arg \max_a \sum_{s'} P(s'|s, a) [R(s, a, s') + \gamma V^\pi(s')]$$

Input

- ▶ Current policy π
- ▶ Value function V^π

Output

- ▶ Improved policy π'
- ▶ Greedy with respect to V^π

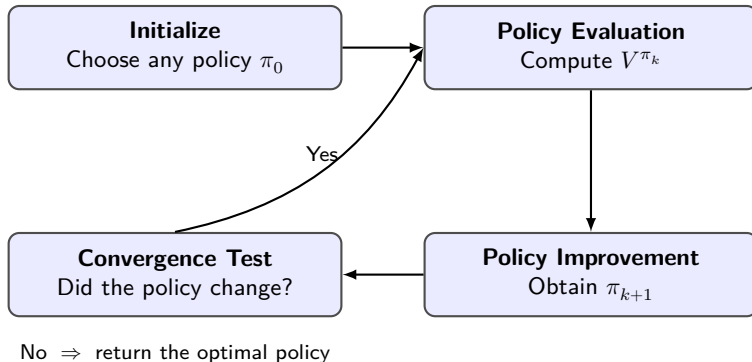
Key Idea

In every state, choose the action that maximizes the expected immediate reward plus the discounted value of the next state.

Policy Iteration

Idea

Policy Iteration alternates between **policy evaluation** and **policy improvement** until the policy no longer changes.



Key Idea

Each iteration produces a policy that is at least as good as the previous one. The algorithm stops when no further improvement is possible.

Value Iteration

Motivation

Policy Iteration alternates between **policy evaluation** and **policy improvement**. However, fully evaluating the policy before every improvement can be computationally expensive.

Key Idea

Instead of completely evaluating the current policy, **Value Iteration** performs a single Bellman optimality update and immediately improves the value estimates.

$$V_{k+1}(s) = \max_a \sum_{s'} P(s'|s, a) [R(s, a, s') + \gamma V_k(s')]$$

Key Idea

Value Iteration combines **policy evaluation** and **policy improvement** into a single update rule.

Policy Iteration vs. Value Iteration

Aspect	Policy Iteration	Value Iteration
Main idea	Alternate evaluation and improvement	Directly update values using the Bellman optimality equation
Value updates	Fully evaluate the current policy	One Bellman update per iteration
Policy updates	After each evaluation	Only after convergence
Stored quantities	Policy and value function	Value function only
Per-iteration cost	Higher	Lower
Number of iterations	Usually fewer	Usually more
Convergence	To the optimal policy	To the optimal value function, then extract the optimal policy
Typical use	Small to medium MDPs	Large MDPs where full evaluation is expensive

Contents

Foundations of Reinforcement Learning

Finding the Optimal Policy

Learning Without a Model

Limitations of Dynamic Programming

Assumption of Dynamic Programming

Policy Iteration and Value Iteration assume that the agent has a complete model of the environment.

Known Information

- ▶ State space S
- ▶ Action space A
- ▶ Transition probabilities

$$P(s' \mid s, a)$$

- ▶ Reward function

$$R(s, a, s')$$

Reality

- ▶ Unknown environment
- ▶ Unknown dynamics
- ▶ Unknown rewards
- ▶ The agent only observes its own experience

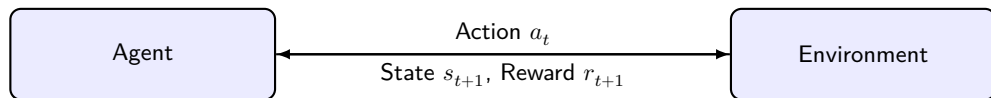
Key Question

How can an agent learn an optimal policy **without knowing the environment beforehand?**

Learning Through Interaction

Key Idea

When the model is unknown, the agent cannot plan ahead. Instead, it must learn by repeatedly interacting with its environment.



Experience

After each interaction, the agent collects one experience:

$$(s_t, a_t, r_{t+1}, s_{t+1})$$

These experiences are gradually used to improve the policy.

Key Difference

Dynamic Programming learns from a **known model**.

Reinforcement Learning learns directly from **experience**.

The Exploration–Exploitation Dilemma

A Fundamental Question

If the agent does not know the environment, how should it choose its next action?

Exploration

Try actions that have not been sufficiently explored.

Goal: Gather new information about the environment.

Risk: The action may lead to a poor reward.

Exploitation

Choose the action that currently appears to be the best.

Goal: Maximize the immediate expected reward.

Risk: A better action may never be discovered.

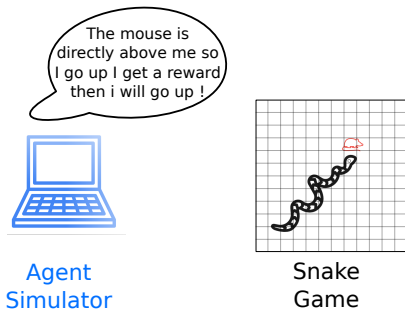
Example

Imagine choosing a restaurant in a new city.

Should you revisit your favorite restaurant (**exploitation**), or try a new one that could be even better (**exploration**)?

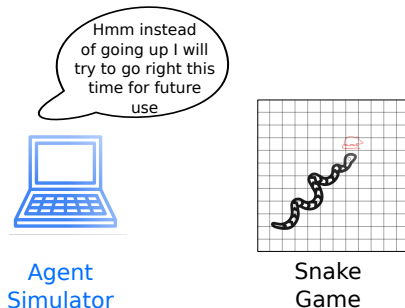
Example: Exploration vs. Exploitation

Exploitation



Choose the action that is currently believed to give the highest reward.

Exploration



Try a different action to discover whether it leads to an even better long-term strategy.

Key Observation

A good reinforcement learning agent must balance **exploitation** (using current knowledge) and **exploration** (acquiring new knowledge).

The ε -Greedy Policy

Idea

Most of the time, choose the action that currently appears to be the best. Occasionally, choose a random action to continue exploring.

$$\pi(a|s) = \begin{cases} \text{Best action,} & \text{with probability } 1 - \varepsilon, \\ \text{Random action,} & \text{with probability } \varepsilon. \end{cases}$$

Small ε

- ▶ Mostly exploit.
- ▶ Little exploration.

Large ε

- ▶ Explore more often.
- ▶ Learn more about the environment.

Typical Choice

A common value is $\varepsilon = 0.1$, meaning the agent explores about 10% of the time and exploits the remaining 90%.

Types of Model-Free Reinforcement Learning

Learning Without a Model

When the transition probabilities and reward function are unknown, reinforcement learning algorithms learn directly from experience.

Model-Free Prediction

Objective:

Estimate the value of a *fixed policy*.

Typical algorithms:

- ▶ Monte Carlo Prediction
- ▶ First-Visit Monte Carlo
- ▶ Every-Visit Monte Carlo

Model-Free Control

Objective:

Learn an *optimal policy* directly from experience.

Typical algorithms:

- ▶ Monte Carlo Control
- ▶ SARSA
- ▶ Q-Learning

Connection

Just as Dynamic Programming separates **policy evaluation** from **policy improvement**, model-free reinforcement learning also distinguishes between **prediction** (estimating values) and **control** (learning an optimal policy).

Monte Carlo Prediction

Key Idea

Instead of using the Bellman equation, Monte Carlo methods estimate the value of a state by averaging the returns observed from complete episodes.

$$V(s) \approx \frac{1}{N} \sum_{i=1}^N G_i(s)$$

where $G_i(s)$ is the return observed after visiting state s during episode i .

Advantages

- ▶ No model required.
- ▶ Easy to understand and implement.
- ▶ Uses real experience.

Limitations

- ▶ Must wait until the episode ends.
- ▶ Slow learning for long episodes.
- ▶ Requires episodic tasks.

First-Visit vs. Every-Visit Monte Carlo

Question

During an episode, a state may be visited multiple times. Should all visits be used to estimate its value?

$$A \rightarrow B \rightarrow C \rightarrow B \rightarrow D \rightarrow \text{Goal}$$

First-Visit Monte Carlo

- ▶ Use only the **first** visit to each state.
- ▶ Avoids multiple updates from the same episode.
- ▶ Simpler theoretical analysis.

Best suited for

- ▶ Strongly cyclic environments.
- ▶ When repeated visits could bias a single episode.

Every-Visit Monte Carlo

- ▶ Use **every** visit to each state.
- ▶ Makes use of all available experience.
- ▶ Usually learns faster in practice.

Best suited for

- ▶ Most practical applications.
- ▶ When more samples are desirable.

Temporal Difference Learning

Key Idea

Instead of waiting until the end of an episode, Temporal Difference (TD) learning updates the value estimate **after every interaction**.

$$V(s_t) \leftarrow V(s_t) + \alpha [r_{t+1} + \gamma V(s_{t+1}) - V(s_t)]$$

Monte Carlo

- ▶ Wait until the episode ends.
- ▶ Use the complete return.
- ▶ One update per episode.

Temporal Difference

- ▶ Update after every step.
- ▶ Learn from partial experience.
- ▶ Faster propagation of knowledge.

Temporal Difference Error

$$\delta = r_{t+1} + \gamma V(s_{t+1}) - V(s_t)$$

The TD error measures how surprising the new experience is.

State Values vs. Action Values

A Limitation of State Values

Knowing that a state is valuable does not directly tell the agent which action it should take.

State Value Function

$$V(s)$$

Measures how good it is to be in state s .

Question:

Once we are in s , which action should we choose?

Action Value Function

$$Q(s, a)$$

Measures how good it is to take action a in state s .

The best action is simply

$$\arg \max_a Q(s, a).$$

Key Idea

Instead of learning how good each **state** is, Q-learning learns how good each **state-action pair** is.

Why Learn Action Values?

Coffee Robot Example

The robot is currently in the **Kitchen** and must decide what to do next.

Using the State Value

Suppose we know

$$V(\text{Kitchen}) = 12.$$

This tells us that the Kitchen is a good state.

But...

Should the robot

- ▶ Go to the Office?
- ▶ Return to Storage?
- ▶ Stay where it is?

The value alone cannot answer this.

Using Action Values

Instead, learn

$$Q(\text{Kitchen}, \text{Office}) = 18$$

$$Q(\text{Kitchen}, \text{Storage}) = 7$$

$$Q(\text{Kitchen}, \text{Stay}) = 2$$

The best action is immediately

$$\arg \max_a Q(s, a).$$

The Action-Value Function

Definition

The action-value function, denoted by $Q(s, a)$, is the expected return obtained by taking action a in state s , and then following the policy thereafter.

$$Q^\pi(s, a) = \mathbb{E}[G_t \mid S_t = s, A_t = a]$$

Input

► Current state & Chosen action

Output

► Expected future return

Interpretation

A larger $Q(s, a)$ indicates that action a is expected to produce a higher long-term reward when taken from state s .

Relationship Between V and Q

The state-value function can be obtained from the action-value function:

$$V^\pi(s) = \sum_a \pi(a \mid s) Q^\pi(s, a).$$

Bellman Optimality Equation for Action Values

Goal

Rather than evaluating a fixed policy, we seek the **optimal action-value function** $Q^*(s, a)$.

$$Q^*(s, a) = \sum_{s'} P(s'|s, a) \left[R(s, a, s') + \gamma \max_{a'} Q^*(s', a') \right]$$

Interpretation

- ▶ Take action a in state s .
- ▶ Receive an immediate reward.
- ▶ Continue with the best possible action thereafter.

Why No Expectation Equation?

An expectation equation for $Q^\pi(s, a)$ also exists. However, our objective is to learn the **optimal policy**, so we directly focus on the optimality equation.

Key Observation

The maximization over a' is the key idea behind **Q-learning**: after taking one action, always assume the best possible action will be chosen next.

Q-Learning

Key Idea

Q-Learning learns the optimal action-value function directly from experience, without requiring a model of the environment.

Action Selection

At each interaction, the agent chooses an action using an ϵ -greedy policy:

- ▶ With probability $1 - \epsilon$, exploit the best known action.
- ▶ With probability ϵ , explore a random action.

Learning

After executing the action, the agent observes:

- ▶ the immediate reward,
- ▶ the next state,

then updates the corresponding **state-action value**.

Key Observation

Over many interactions, the estimated action values become more accurate, allowing the agent to progressively improve its decisions.

Q-Learning Update Rule

Updating the Action Value

After each interaction, only the experienced state-action pair is updated.

$$Q(s_t, a_t) \leftarrow Q(s_t, a_t) + \alpha \left(r_{t+1} + \gamma \max_a Q(s_{t+1}, a) - Q(s_t, a_t) \right)$$

Current Estimate

- ▶ $Q(s_t, a_t)$
- ▶ Learning rate α

Target

- ▶ Immediate reward r_{t+1}
- ▶ Best future value $\max_a Q(s_{t+1}, a)$

Interpretation

The update moves the current estimate toward a better estimate obtained from the latest experience.

Decision-Making Algorithms in Artificial Intelligence

Category	Algorithms	Main Idea	When to Use	Environment
Classical Search	DFS, BFS, Dijkstra, A*	Find a path from an initial state to a goal.	Single-agent path planning.	Known map, no opponent.
Adversarial Search	Minimax, Alpha-Beta, MCTS	Choose actions against an intelligent opponent.	Games and competitive decision making.	Known rules, adversarial.
Reinforcement Learning (Known Model)	Policy Iteration, Value Iteration	Compute the optimal policy using the environment model.	Planning when transitions and rewards are known.	Known dynamics.
Reinforcement Learning (Unknown Model)	Monte Carlo, Temporal Difference, Q-Learning	Learn the optimal policy directly from experience.	Unknown environments requiring exploration.	Unknown dynamics.

Big Picture

Although these algorithms appear different, they all solve the same fundamental problem:

How should an intelligent agent choose its next action?

The main differences lie in the available information (known model, unknown model, or opponent) and in how decisions are computed.

Beyond This Course: Deep Q-Learning

Motivation

Classical Q-Learning stores one value for every **state-action pair**.

This becomes impractical when the number of states is very large (e.g., images, videos, robotics).

Classical Q-Learning

$$Q(s, a)$$

- ▶ Lookup table
- ▶ Suitable for small state spaces
- ▶ Fast and easy to understand

Deep Q-Learning

$$Q(s, a; \theta)$$

A neural network approximates the action-value function.

- ▶ Large or continuous state spaces
- ▶ Generalizes to unseen states
- ▶ Learns directly from raw observations

Why Is It Important?

Deep Q-Learning (DQN) enabled reinforcement learning to solve challenging problems such as playing Atari games directly from pixels and inspired many modern deep reinforcement learning algorithms.

Pseudo-code: Policy Evaluation

Policy Evaluation

Function PolicyEvaluation(π, γ, θ)

Initialize $V(s) \leftarrow 0$ for every state s

Repeat

$\Delta \leftarrow 0$

For each non-terminal state s

oldValue $\leftarrow V(s)$

$V(s) \leftarrow 0$

For each action a

For each possible next state s'

$V(s) \leftarrow V(s)$
 $+ \pi(a | s) P(s' | s, a)$
 $\times [R(s, a, s') + \gamma V(s')]$

$\Delta \leftarrow \max(\Delta, |\text{oldValue} - V(s)|)$

Until $\Delta < \theta$

return V

Pseudo-code: Policy Improvement

Policy Improvement

Function PolicyImprovement(V, γ)

Initialize policy π'

For each non-terminal state s

 bestAction $\leftarrow$ None

 bestValue $\leftarrow -\infty$

For each action a

 actionValue $\leftarrow 0$

For each possible next state s'

 actionValue $\leftarrow$ actionValue

$+ P(s' | s, a)$

$\times [R(s, a, s') + \gamma V(s')]$

If actionValue $>$ bestValue

 bestValue $\leftarrow$ actionValue

 bestAction $\leftarrow$ action

$\pi'(s) \leftarrow$ bestAction

return π'

Pseudo-code: Policy Iteration

Policy Iteration

Function PolicyIteration(γ, θ)

Initialize policy π arbitrarily

Initialize $V(s) \leftarrow 0$ for every state s

Repeat

$V \leftarrow \text{PolicyEvaluation}(\pi, \gamma, \theta)$

$\text{newPolicy} \leftarrow \text{PolicyImprovement}(V, \gamma)$

If $\text{newPolicy} = \pi$
 break

$\pi \leftarrow \text{newPolicy}$

return (π, V)

Pseudo-code: Value Iteration

Value Iteration

Function ValueIteration(γ, θ)

Initialize $V(s) \leftarrow 0$ for every state

Repeat

$\Delta \leftarrow 0$

For each non-terminal state s

oldValue $\leftarrow V(s)$

bestValue $\leftarrow -\infty$

For each action a

actionValue $\leftarrow 0$

For each next state s'

actionValue $\leftarrow$ actionValue
+ $P(s'|s, a)$
 $\times [R(s, a, s') + \gamma V(s')]$

bestValue $\leftarrow$

max(bestValue, actionValue)

$V(s) \leftarrow$ bestValue

$\Delta \leftarrow$

max($\Delta, |oldValue - V(s)|$)

Until $\Delta < \theta$

For each non-terminal state s

$\pi(s) \leftarrow$

$\arg \max_a$
 $\sum_{s'} P(s'|s, a)$
 $\times [R(s, a, s') + \gamma V(s')]$

return (π, V)

Pseudo-code: First-Visit Monte Carlo

First-Visit Monte Carlo

Function FirstVisitMC(π, γ)

Initialize $V(s) \leftarrow 0$ and Returns(s)

For each episode

 Generate an episode following policy π

$G \leftarrow 0$

For each step from the end to the beginning

$G \leftarrow \gamma G + r$

If first visit to state s

 Append G to Returns(s)

$V(s) \leftarrow \text{Average}(\text{Returns}(s))$

return V

Pseudo-code: Every-Visit Monte Carlo

Every-Visit Monte Carlo

```
Function EveryVisitMC( $\pi, \gamma$ )  
  Initialize  $V(s) \leftarrow 0$  and Returns( $s$ )  
  For each episode  
    Generate an episode following policy  $\pi$   
     $G \leftarrow 0$   
    For each step from the end to the beginning  
       $G \leftarrow \gamma G + r$   
      Append  $G$  to Returns( $s$ )  
       $V(s) \leftarrow \text{Average}(\text{Returns}(s))$   
  return  $V$ 
```

Pseudo-code: Temporal Difference Learning

Temporal Difference Learning

Function TDLearning(π, α, γ)

Initialize $V(s) \leftarrow 0$

For each episode

 Initialize starting state s

While state is not terminal

 Choose action according to policy π

 Execute action

 Observe reward r and next state s'

$V(s) \leftarrow V(s)$
 $+ \alpha [r + \gamma V(s') - V(s)]$

$s \leftarrow s'$

return V

Pseudo-code: Q-Learning

Q-Learning

Function QLearning($\alpha, \gamma, \varepsilon$)

Initialize $Q(s, a) \leftarrow 0$

For each episode

 Initialize starting state s

While state is not terminal

 Choose action using an ε -greedy policy

 Execute action

 Observe reward r and next state s'

$Q(s, a) \leftarrow Q(s, a)$
 $+ \alpha [r + \gamma \max_{a'} Q(s', a') - Q(s, a)]$

$s \leftarrow s'$

Derive policy $\pi(s) \leftarrow \arg \max_a Q(s, a)$

return (Q, π)

Pseudo-code: Deep Q-Learning (DQN)

Deep Q-Learning (DQN)

Function DQN($\alpha, \gamma, \varepsilon$)

Initialize Q-network with random weights θ

Initialize target network with weights θ^-

Initialize replay memory

For each episode

 Initialize starting state s

While state is not terminal

 Choose action using an ε -greedy policy

 Execute action

 Observe reward r and next state s'

 Store (s, a, r, s') in replay memory

 Sample a random mini-batch from replay memory

 Update the Q-network using gradient descent

 Periodically copy θ to θ^-

 // Update target network

$s \leftarrow s'$

Derive policy $\pi(s) \leftarrow \arg \max_a Q(s, a; \theta)$

return (Q, π)